

A Sprint de um QA

Qualidade de software na prática

// Dia 1: Fundamentos, requisitos e testes manuais

> Lucas da Cunha Rodrigues

> UDESC · 06/10

[ALERTA DE SISTEMA]



**O Relógio
Está Correndo**
HH:MM:SS



**Você acabou de entrar no time de
Qualidade.**

O Produto:

Inscrevi (Sistema de inscrição em minicursos de eventos acadêmicos).



O Cenário:

A Sprint termina hoje.

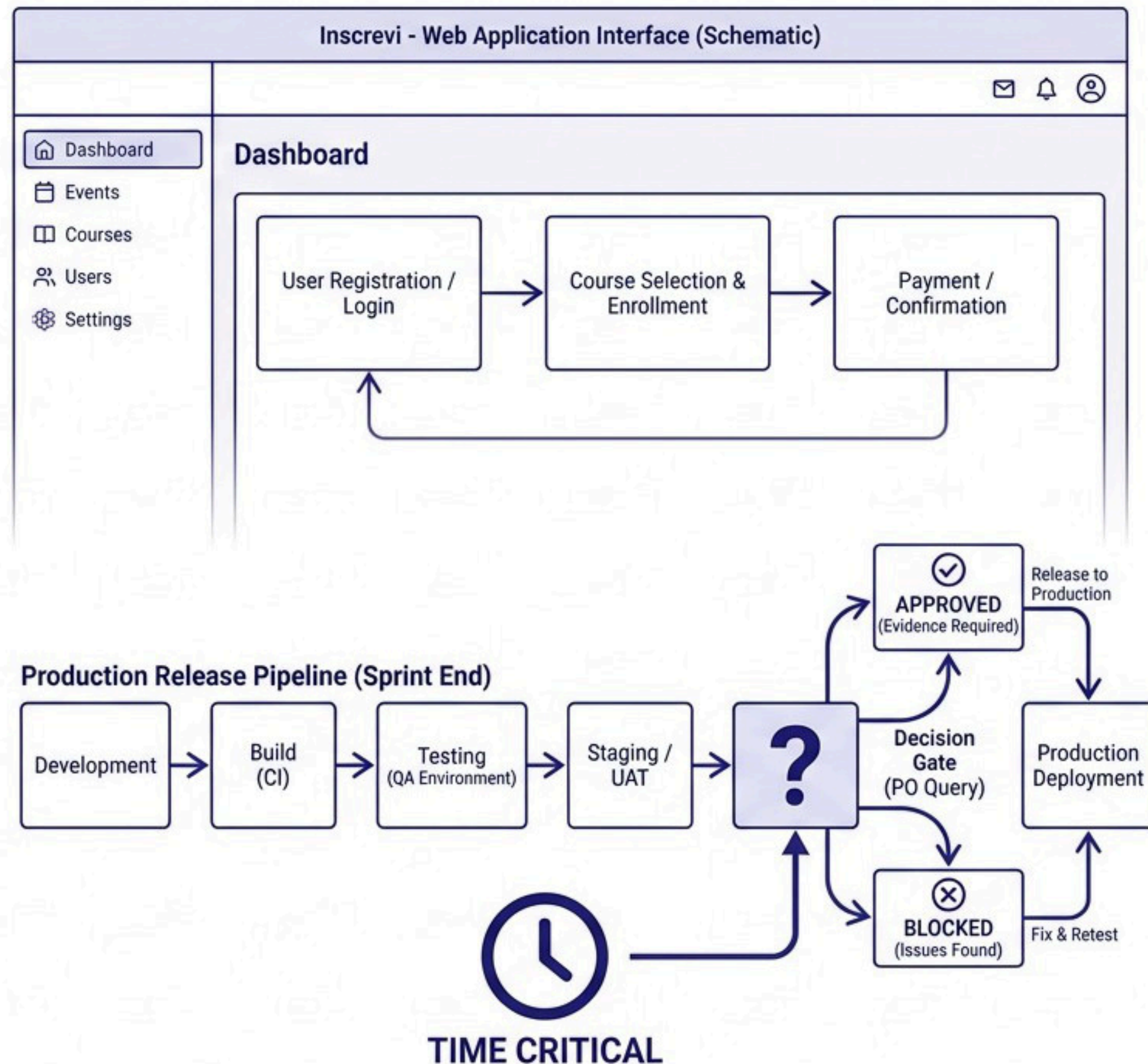


A Pergunta do PO:

Dá para liberar para produção?

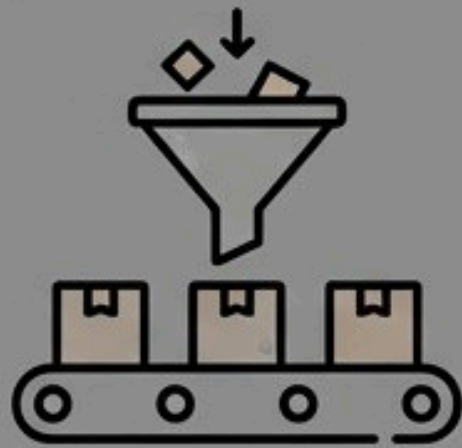


A resposta só sai no fim da noite.
E precisa de evidências.



A Evolução da Qualidade (CTFL 1.1)

O Tester Tradicional



- Filtro de fim de linha.
- Executa software pronto, encontra falhas.
- Foco: Teste dinâmico (depois do código).

O QA Moderno



- Garantia e Prevenção (Shift-Left).
- Revisa a ideia, questiona a regra, previne o defeito.
- Foco: Teste estático (antes do código) + Dinâmico.

Testar não é depurar. Qualidade é construída, não adicionada no final.

Anatomia de um Problema (Por que testamos?)

O Erro (Ação)



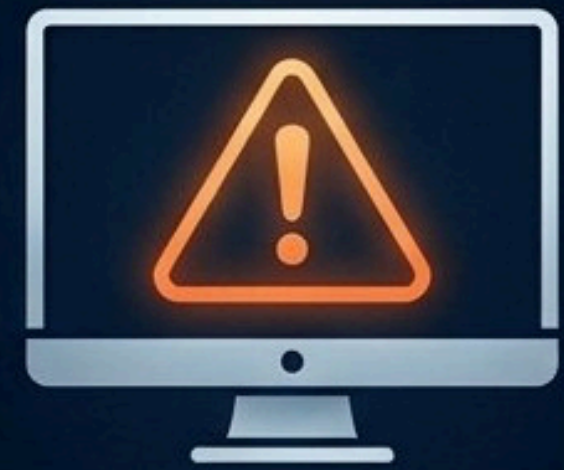
Ação humana equivocada.
(Ex: O Dev entende a regra de limite errado).

O Defeito (Artefato)



O problema escondido no código ou requisito.
(Ex: O código aceita > 10).

A Falha (Comportamento)

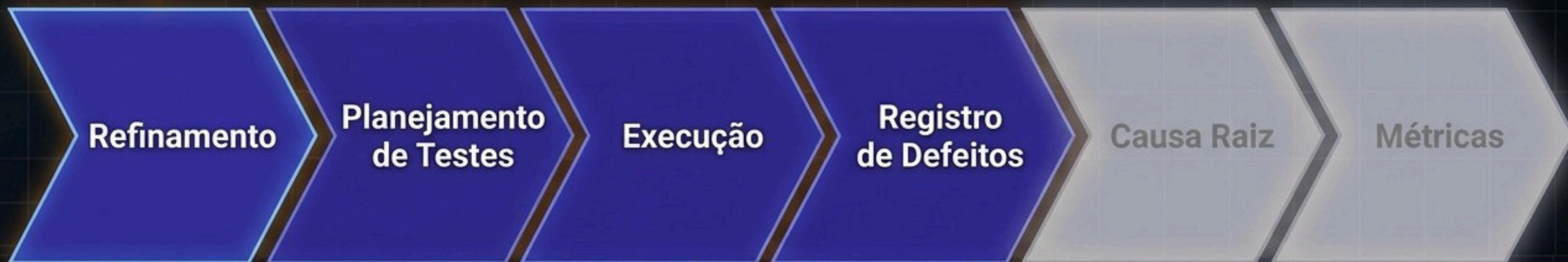


O sistema quebra na mão do usuário. (Ex: Usuário consegue burlar o limite).

O QA **rastreia a falha** para achar o defeito e entender o erro (Causa Raiz).

O Fluxo de Qualidade na Sprint

Você está aqui: Testar começa antes do código.



A Régua do Refinamento: Matriz INVEST

1



[I] Independente

Pode ser entregue sozinha?

2



[N] Negociável

Diz o quê, não o como?

3



[E] Estimável

O time sabe o suficiente para fazer?

4



[S] Small (Pequena)

É uma coisa só?

5



[S] Small (Pequena)

É uma coisa só?

6



[T] Testável

Consigo escrever um teste com resultado claro?



Sinais de Alerta

Palavras vagas (rapidamente, razoável), números mágicos, múltiplas funcionalidades (E também...).

Cortando o Infinito: Técnicas de Design (CTFL 4.2)

Particionamento de Equivalência



Agrupe o que o sistema trata igual.
Um teste por grupo.
(Teste exaustivo é impossível - Princípio 2).

Análise de Valor Limite



Defeitos se escondem nas bordas.
Teste o limite e o vizinho.
A técnica mais rentável.

Mapeando Regras Complexas (CTFL 4.2)

Tabela de Decisão

[Logado?]	[Já Inscrito?]	[Tem Vaga?]	
✓	✓	✓	[Verde: Aceito]
✓	✓	✓	
✓	✗	✗	
✗	✓	✗	[Vermelho: Negado]
✗	✗	✓	

Para regras que combinam condições.
Cada coluna vira um caso de teste.

Transição de Estado



Para ciclos de vida. Teste as
transições válidas e as inválidas.

O Plano de Batalha & BDD

Risk Matrix (CTFL 5.2)

Probabilidade de Falhar	high			Alta Prioridade
	low			
		low	Impacto no Negócio	high

O plano é uma decisão, não uma lista.
Priorize o risco.

Anatomia do Cenário BDD

```
1 DADO [o estado de partida real]
2 QUANDO [uma ação única]
3 ENTÃO [um resultado observável e inquestionável]
```

Outra pessoa deve conseguir executar e chegar ao mesmo resultado sem perguntar nada.

Duas Frentes de Batalha: Tela x API

A Vitrine (Interface)

Pode esconder botões e mascarar regras.
Lenta, instável para automatizar.

Um teste pode passar na API e falhar na tela. Ou o contrário.

O Motor (API)

Aonde a regra de negócio realmente mora. **Rápida, estável.**
Base da pirâmide de testes.



⚡ ALÉM DO SCRIPT: TESTE EXPLORATÓRIO (CTFL 4.4)

CORE CONCEPT

Aprender, projetar e executar simultaneamente.
Acha o que ninguém pensou em escrever.

THE CHARTER (A MISSÃO)

- 🎯 **Alvo:** Qual área do sistema explorar.
- ⚙️ **Recursos:** Quais dados/contas usar.
- 🕒 **Timebox:** Ex: 10 minutos de foco total.



A ARMA SECRETA

SUPOSIÇÃO DE ERRO

"Se eu fosse o dev construindo isso às pressas, onde eu erraria?"

A Anatomia do Relato



- ✓ 1. Título auto-explicativo.
- ✓ 2. Ambiente/Versão.
- ✓ 3. Passos exatos + Dados usados.
- ✓ 4. Esperado vs Obtido.
- ✓ 5. Evidência irrefutável (Print/Log).

O Kanban



O Placar da Sprint (Métricas)

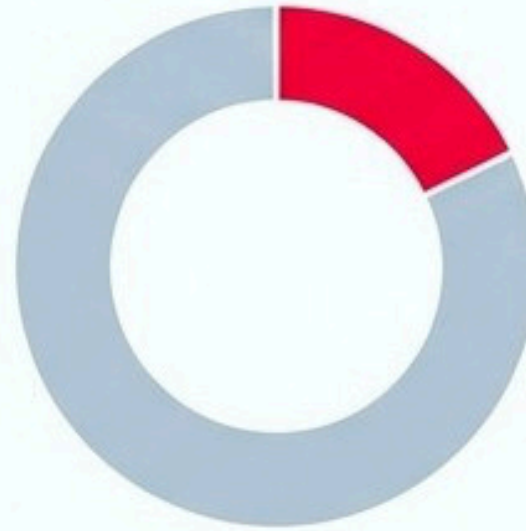
Defeitos Abertos x Corrigidos

15 / 25



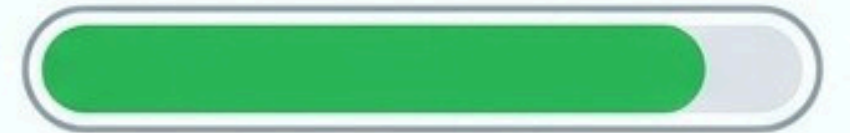
Defeitos Críticos / Bloqueadores

3



Taxa de Sucesso dos Testes

88%



**Métrica serve para decidir, não para punir.
Defeito em homologação custa pouco. Em produção custa a confiança.**